Practical - 11

Student Management CRUD with Search and Filter

Problem Definition: Create a student management module to add, view, edit, delete, search and filter student records. All database operations must use MYSQLi statements and clear success/failure messages.

A. Theory

1. Are SELECT, INSERT, UPDATE, and DELETE operations working? Yes — all four CRUD operations are implemented:

SELECT → students.php fetches and displays all students from the students table.
INSERT → add_student.php adds a new student with duplicate check and password hashing.
UPDATE → edit_student.php pre-fills the form using SELECT, then updates the record on submit.
DELETE → students.php handles ?delete=id in the URL, runs DELETE FROM students WHERE student_id = $id.

2. Is search by name/email/course implemented?

Yes — the search form on students.php uses a LIKE '%search%' query on full_name and email, plus a dropdown filter for course. Both can be used together at the same time.

3. Are delete operations confirmed before execution?

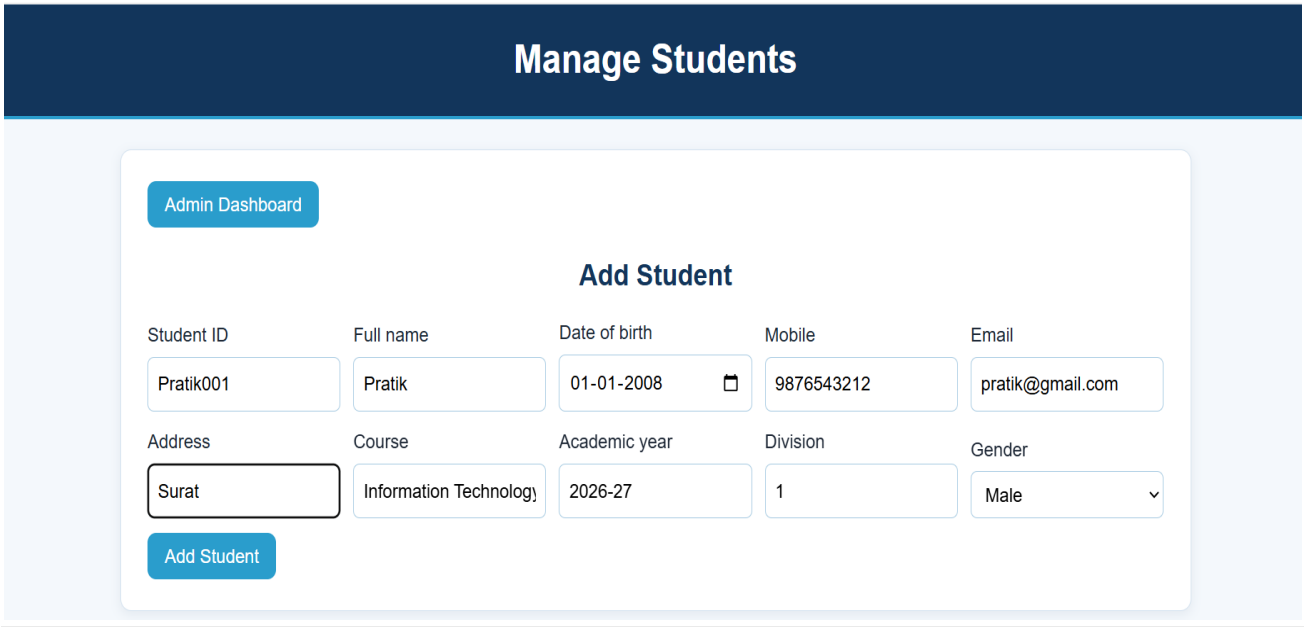
Yes — the Delete button uses JavaScript's built-in onclick="return confirm('Are you sure you want to delete this student?')". If the user clicks Cancel, the deletion does not happen.

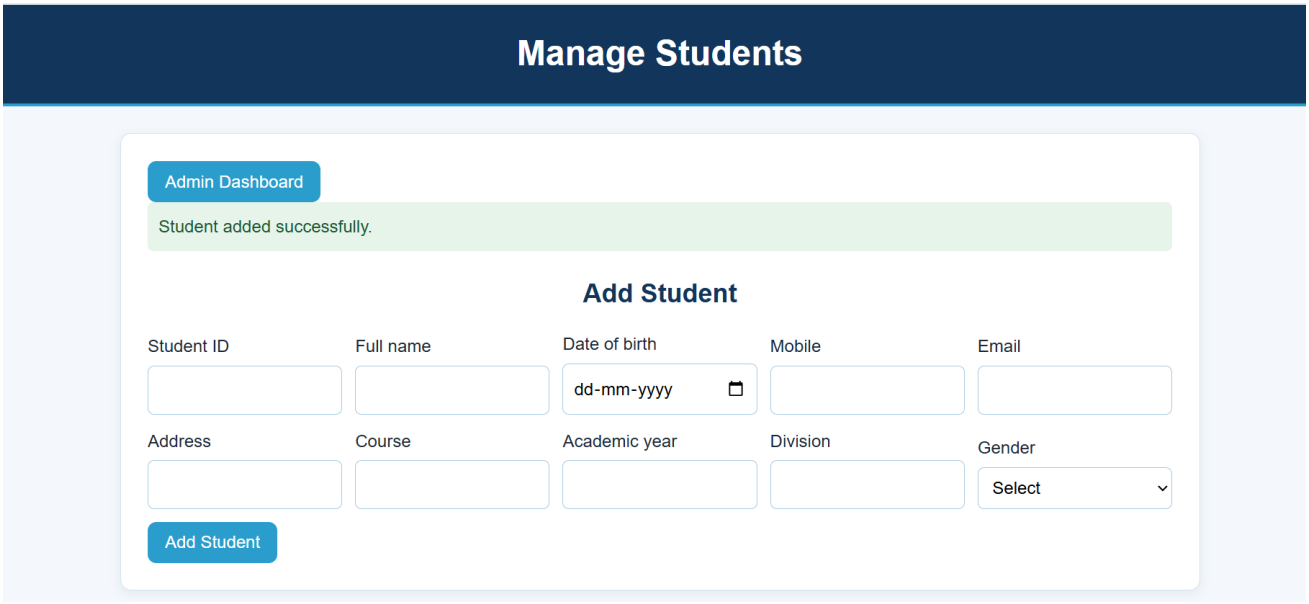
B. Implementation

File	What It Does
db_connect.php	Reused from Practical 9 — basic MySQLi

students.php	Lists all students, handles
add_student.php	Form to INSERT a new student
edit_student.php	Pre-fills form via SELECT, updates via UPDATE

C. Evidence of Work (Screenshots)





Krish123	Krish	krish@gmail.com	Computer Engineering	Second Year / 1	Edit Delete
Niket123	Niket	niket123@gmail.com	Computer Engineering	First Year / 1	Edit Delete
Pratik001	Pratik	pratik@gmail.com	Information Technology	2026-27 / 1	Edit Delete
Spandan037	Spandan Kakadiya	spandan@gmail.com	Computer Engineering	Second Year / 1	Edit Delete

D. GitHub Repository Link:

https://github.com/25dce037-tech/Student-Hub-Portal.git

E. Challenges Faced:

1.Building dynamic SQL queries — Adding WHERE 1=1 to the base query made it easy to optionally append search and filter conditions without breaking the syntax.
2.Pre-filling the edit form — Fetching the current row via SELECT and displaying its values as value="..." in the HTML form required careful use of htmlspecialchars() to avoid breaking the HTML if a student's name has quotes.
3.Preventing accidental deletes — A direct link without confirmation would delete data immediately, so onclick="return confirm(...)" was added as a safety net.

F. Learning Outcomes:

1.Learned all four CRUD operations (Create, Read, Update, Delete) using MySQLi procedural style.
2.Understood how to build dynamic SQL queries with optional WHERE clauses for search and filter.
3.Learned how LIKE '%keyword%' works for pattern-based searching in MySQL.
4.Understood the importance of confirming destructive actions before executing them.
5.Learned how to pre-fill HTML forms from database data to create a smooth edit experience.

Faculty Remarks: